

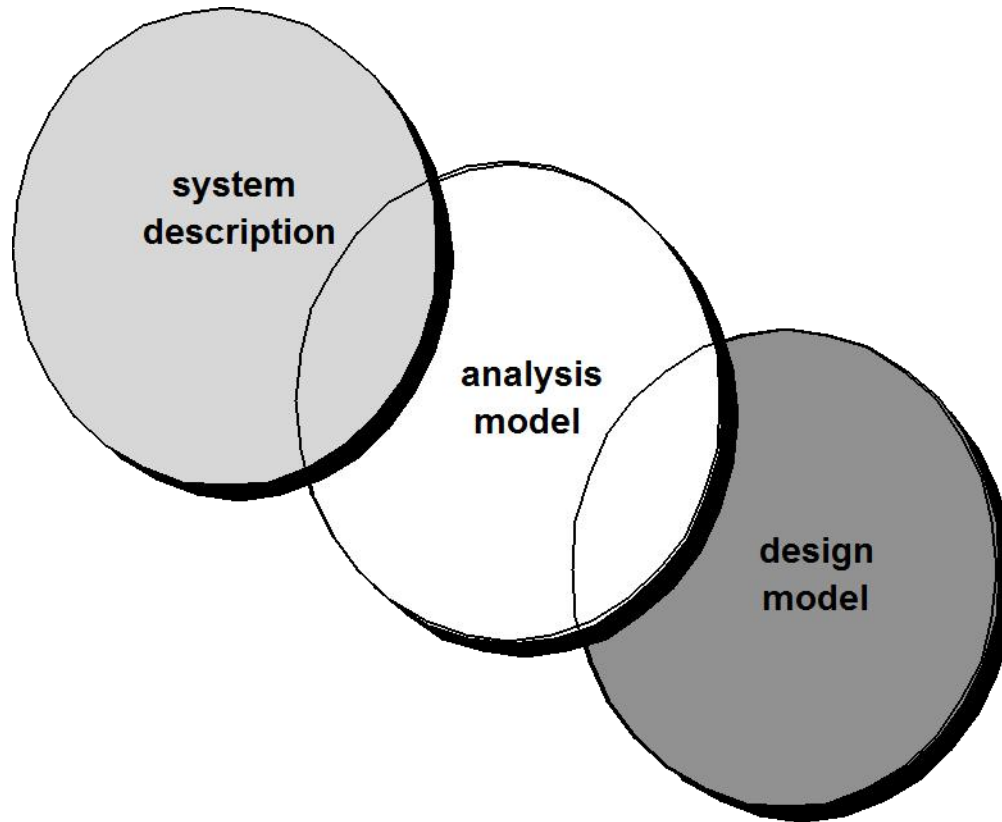
Modeling



Requirements Analysis

- Requirements analysis
 - specifies software's operational characteristics
 - indicates software's interface with other system elements
 - establishes constraints that software must meet
- Requirements analysis allows the software engineer (called an *analyst* or *modeler* in this role) to:
 - elaborate on basic requirements established during earlier requirement engineering tasks
 - build models that depict user scenarios, functional activities, problem classes and their relationships, system and class behavior, and the flow of data as it is transformed.

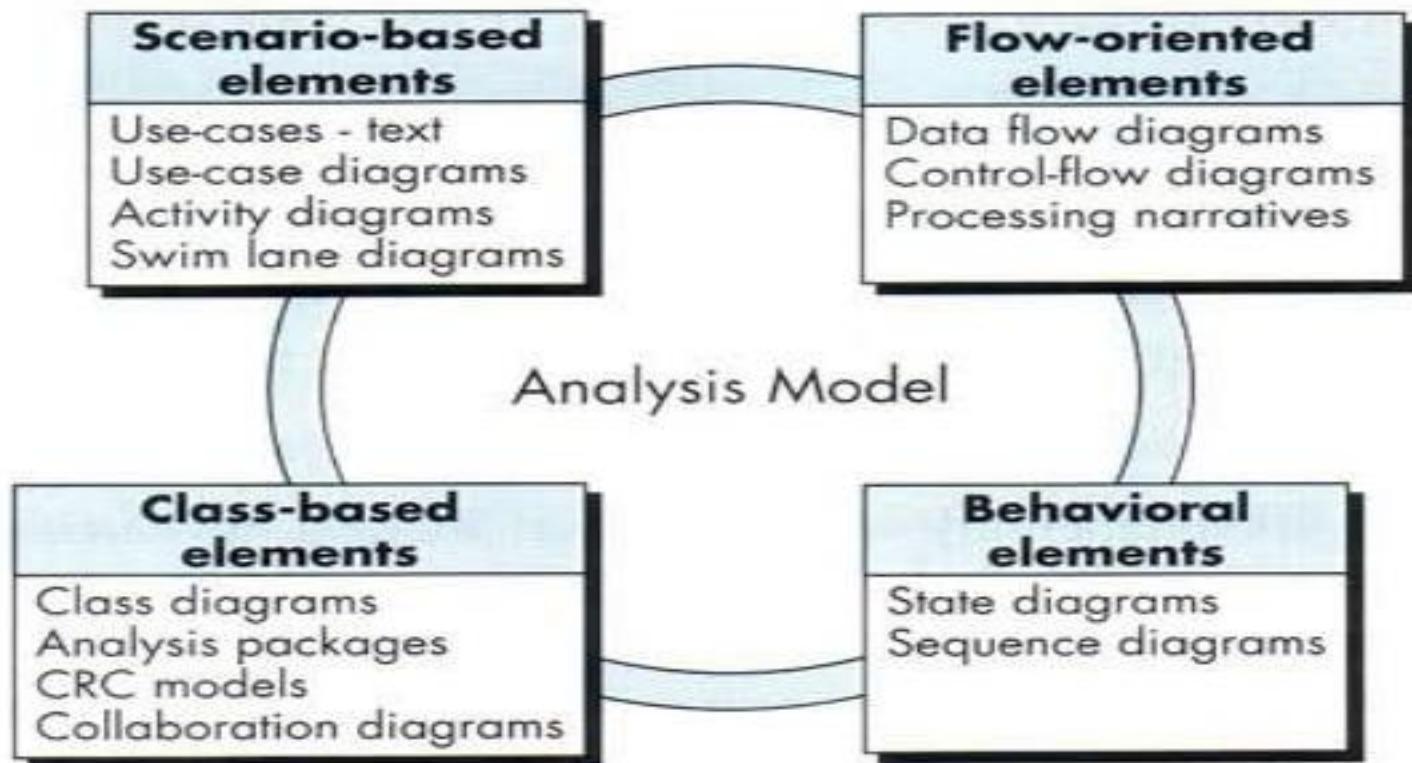
A Bridge



Analysis model

- One view of requirements modeling, called *structured analysis*, considers data and the processes that transform the data as separate entities.
 - Data objects are modeled in a way that defines their attributes and relationships.
 - Processes that manipulate data objects are modeled in a manner that shows how they transform data as data objects flow through the system.
- A second approach to analysis modeled, called *object-oriented analysis*, focuses on
 - the definition of classes and
 - the manner in which they collaborate with one another to effect customer requirements.

Elements of Analysis model



Elements of Analysis model

Each element of the requirements model presents the problem from a different point of view.

- **Scenario-based** elements depict how the user interacts with the system and the specific sequence of activities that occur as the software is used.
- **Class-based** elements model the objects that the system will manipulate, the operations that will be applied to the objects to effect the manipulation, relationships (some hierarchical) between the objects, and the collaborations that occur between the classes that are defined.
- **Behavioral elements** depict how external events change the state of the system or the classes that reside within it.
- Finally, **flow-oriented** elements represent the system as an information transform, depicting how data objects are transformed as they flow through various system functions.

Scenario-Based Modeling

“[Use-cases] are simply an aid to defining what exists outside the system (actors) and what should be performed by the system (use-cases).”

- (1) What should we write about?**
- (2) How much should we write about it?**
- (3) How detailed should we make our description?**
- (4) How should we organize the description?**

What to Write About?

- **Inception and elicitation**—provide you with the information you'll need to begin writing use cases.
- **Requirements gathering meetings, QFD(Quality Function Deployment), and other requirements engineering mechanisms are used to**
 - identify stakeholders
 - define the scope of the problem
 - specify overall operational goals
 - establish priorities
 - outline all known functional requirements, and
 - describe the things (objects) that will be manipulated by the system.
- To begin developing a set of use cases, **list the functions or activities performed by a specific actor.**

How Much to Write About?

- As further conversations with the stakeholders progress, the requirements gathering team develops use cases for each of the functions noted.
- In general, use cases are written first in an informal narrative fashion.

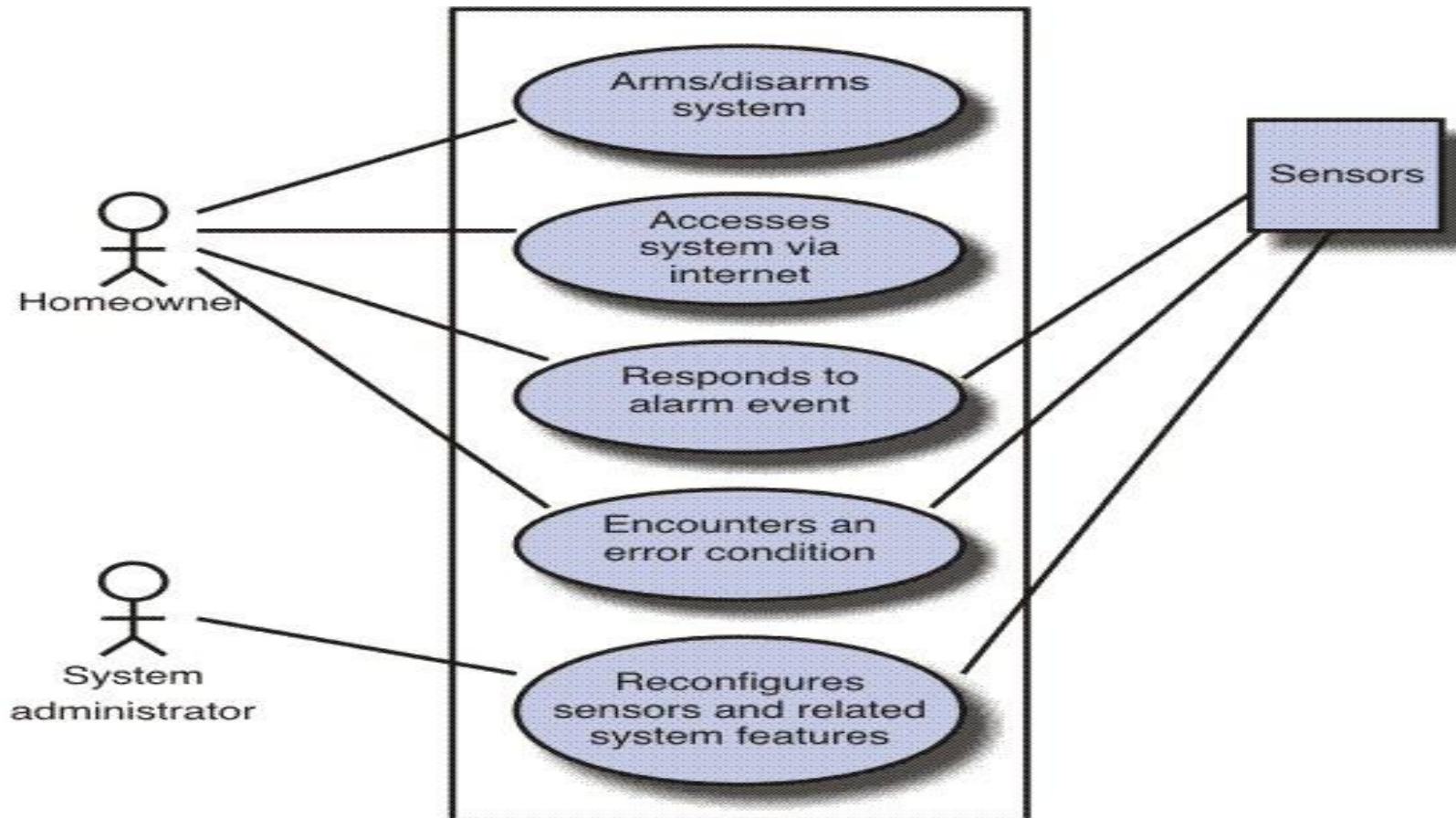
Use-Cases

- a scenario that describes a “thread of usage” for a system
- *actors* represent roles people or devices play as the system functions
- *users* can play a number of different roles for a given scenario

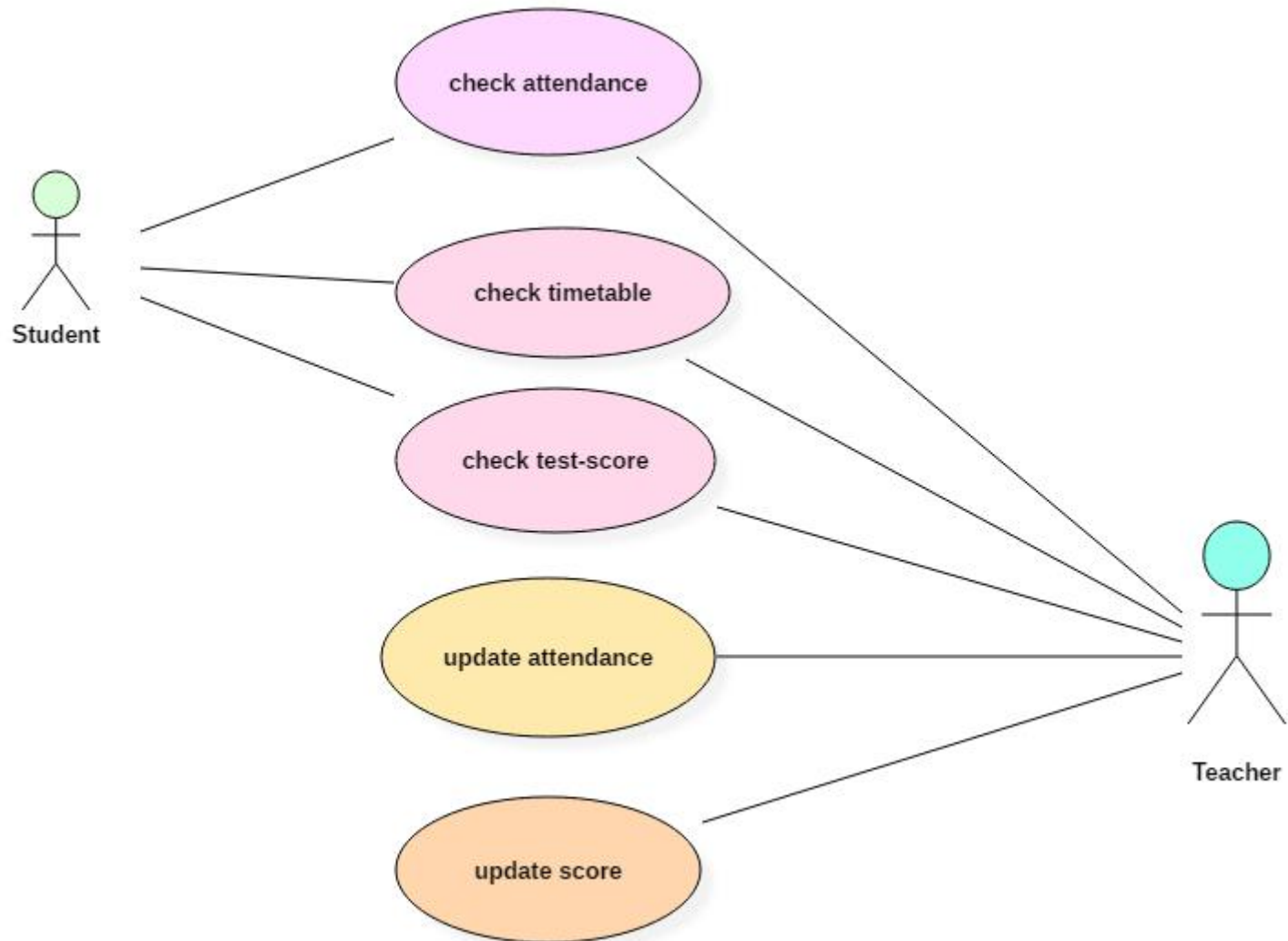
Developing a Use-Case

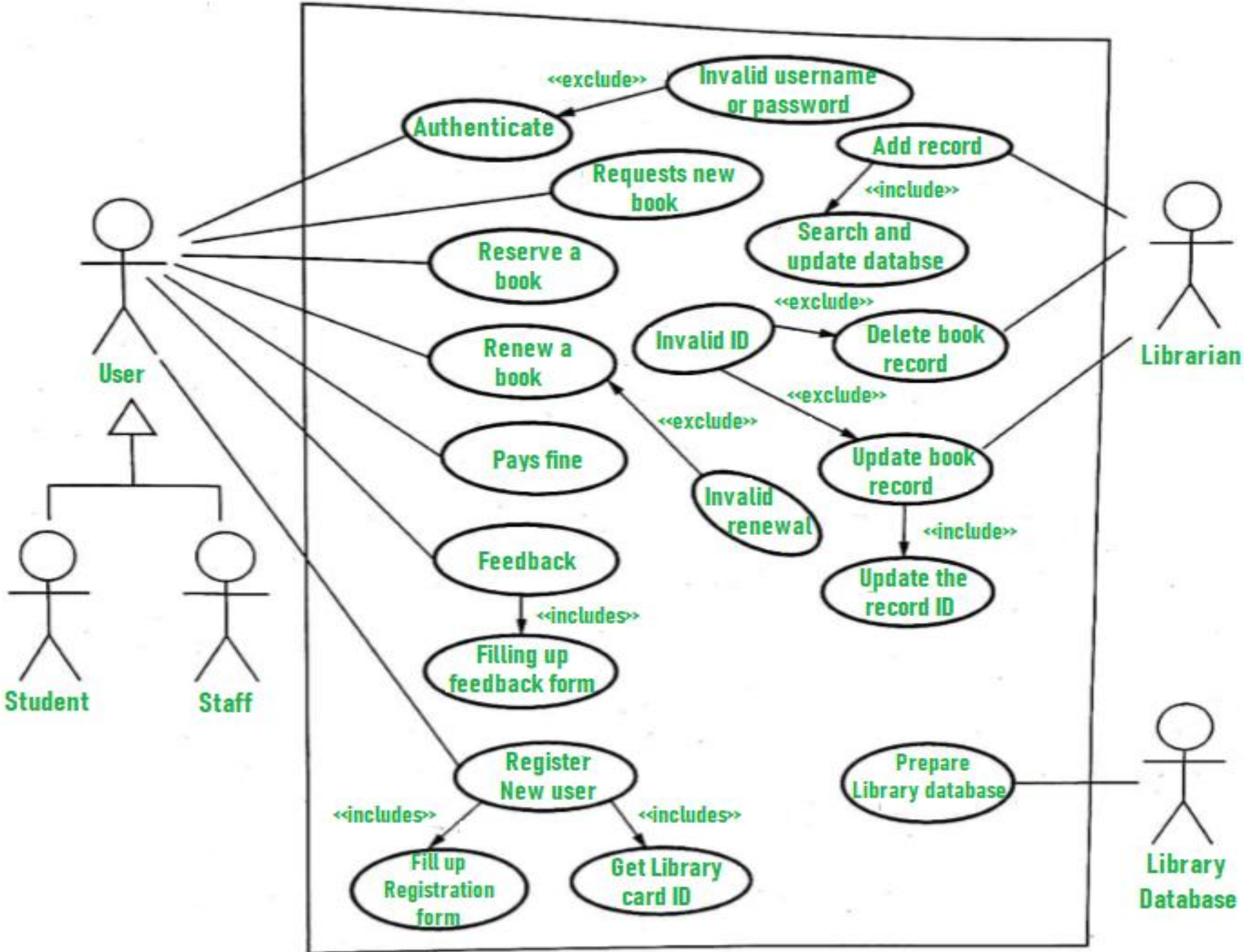
- What are the main tasks or functions that are performed by the actor?
- What system information will the actor acquire, produce or change?
- Will the actor have to inform the system about changes in the external environment?
- What information does the actor desire from the system?
- Does the actor wish to be informed about unexpected changes?

Use-Case Diagram



Example of Use case Diagram

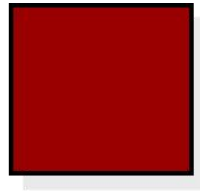




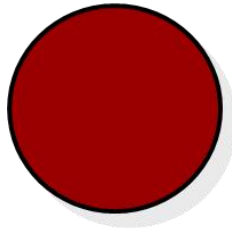
Flow-Oriented Modeling

- Represents how data objects are transformed as they move through the system.
- **data flow diagram (DFD)** is the diagrammatic form that is used.
- Considered by many to be an “old school” approach, but continues to provide a view of the system that is unique—it should be used to supplement other analysis model elements.

Flow Modeling Notation



external entity



process



data flow



data store

External Entity

A producer or consumer of data

Examples: a person, a device, a sensor

Another example: computer-based system

Data must always originate somewhere and must always be sent to something

Process



A data transformer (changes input to output)

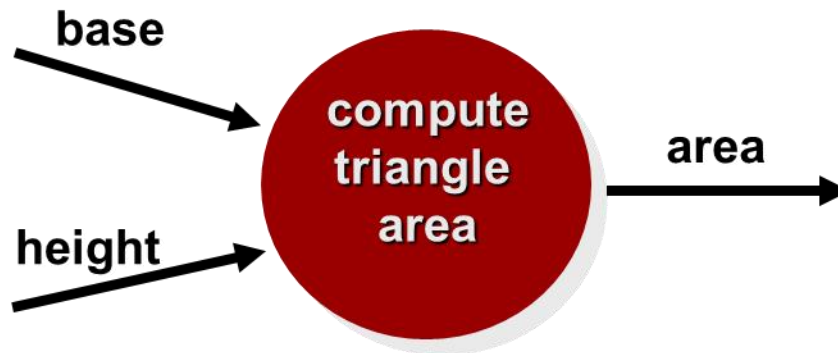
Examples: *compute taxes, determine area, format report, display graph*

Data must always be processed in some way to achieve system function

Data Flow

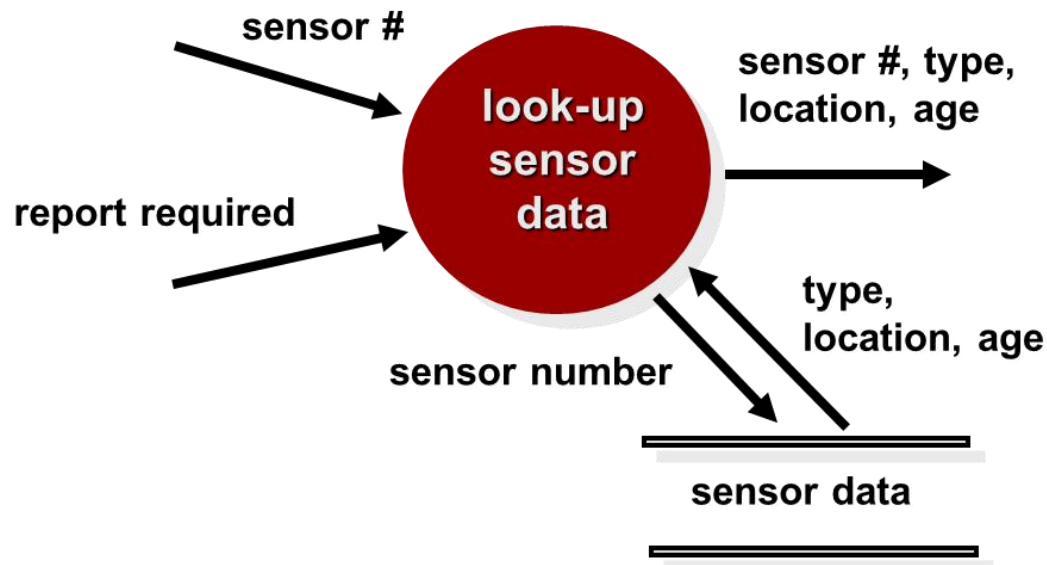


Data flows through a system, beginning as input and transformed into output.



Data Stores

Data is often stored for later use.



Data Flow Diagramming: Guidelines

- all icons must be labeled with meaningful names.
- the DFD evolves through a number of levels of detail.
- always begin with a context level diagram (also called level 0).
- always show external entities at level 0.
- always label data flow arrows.
- do not represent procedural logic.

Data Modeling

- If software requirements include the need to create, extend, or interface with a database or if complex data structures must be constructed and manipulated, the software team may choose to create a data model as part of overall requirements modeling.
- A software engineer or analyst defines all data objects that are processed within the system, the relationships between the data objects, and other information that is pertinent to the relationships.
- The entity-relationship diagram (ERD) addresses these issues and represents all data objects that are entered, stored, transformed, and produced within an application.

What is a Data Object?

- a representation of almost any composite information that must be understood by software.
 - *composite information*—something that has a number of different properties or attributes.
 - width (a single value) would not be a valid data object, but dimensions (incorporating height, width, and depth) could be defined as an object.
- can be an **external entity** (e.g., anything that produces or consumes information), **a thing** (e.g., a report or a display), **an occurrence** (e.g., a telephone call) **or event** (e.g., an alarm), **a role** (e.g., salesperson), **an organizational unit** (e.g., accounting department), **a place** (e.g., a warehouse), or **a structure** (e.g., a file).
- The description of the data object incorporates the data object and all of its attributes.
- A data object encapsulates data only—there is no reference within a data object to operations that act on the data.

Data Objects and Attributes

A data object contains a set of attributes that act as an aspect, quality, characteristic, or descriptor of the object.

object: automobile

attributes:

make

model

body type

price

options code

Data Objects and Object-Oriented Classes— Are They the Same Thing?

- A **data object** defines a composite data item; that is, it incorporates a collection of individual data items (attributes) and gives the collection of items a name (the name of the data object).
- An **object-oriented class** encapsulates data attributes but also incorporates the operations (methods) that manipulate the data implied by those attributes.
- Classes communicate with one another via messages, they can be organized into hierarchies, and they provide inheritance characteristics for objects that are an instance of a class.

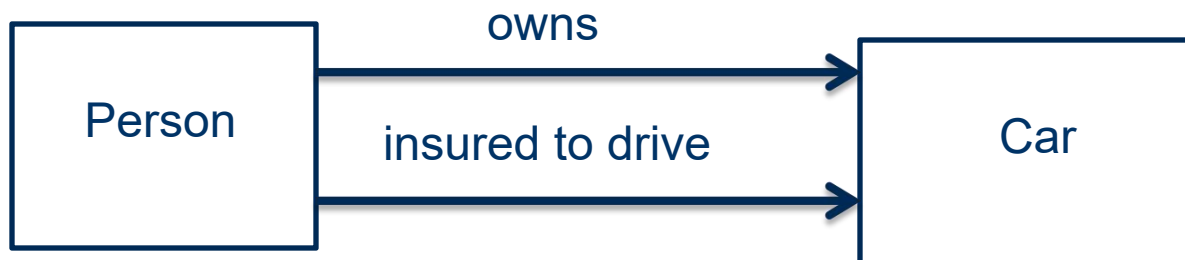
What is a Relationship?

- Data objects are connected to one another in different ways.
 - A connection can be established between **person** and **car** because the two objects are related.
 - A person *owns* a car
 - A person *is insured to drive* a car
- The relationships *owns* and *insured to drive* define the relevant connections between **person** and **car**.
- Several instances of a relationship can exist
- Objects can be related in many different ways

Relationship



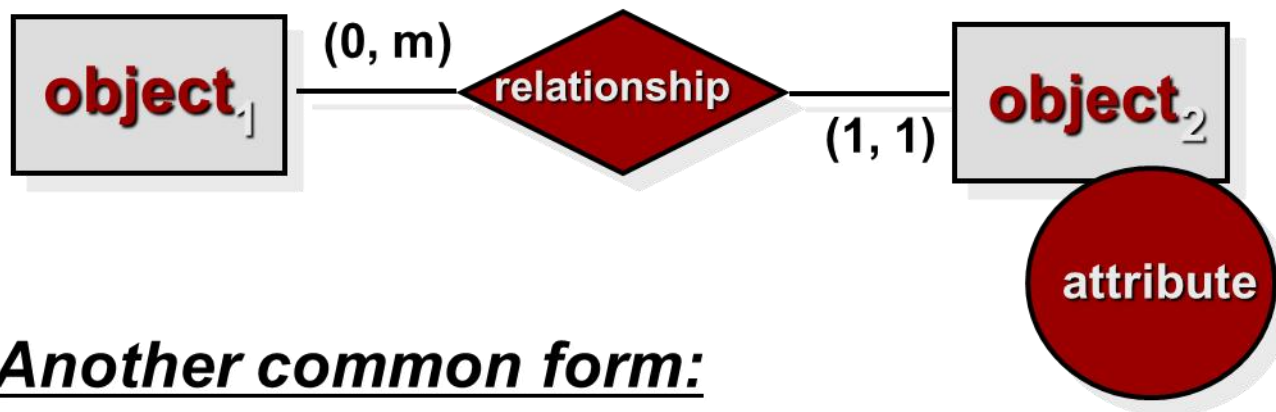
A basic connection between data objects



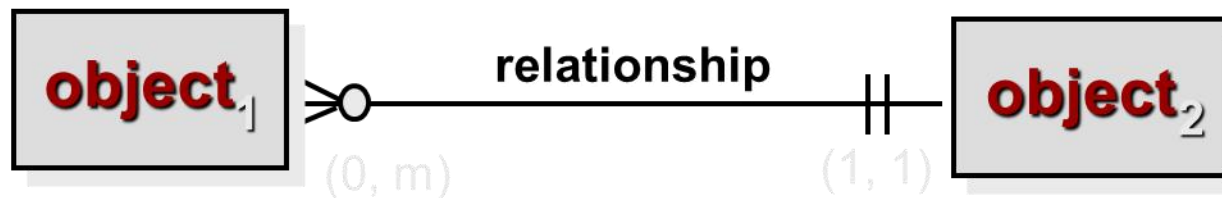
Relationships between data objects

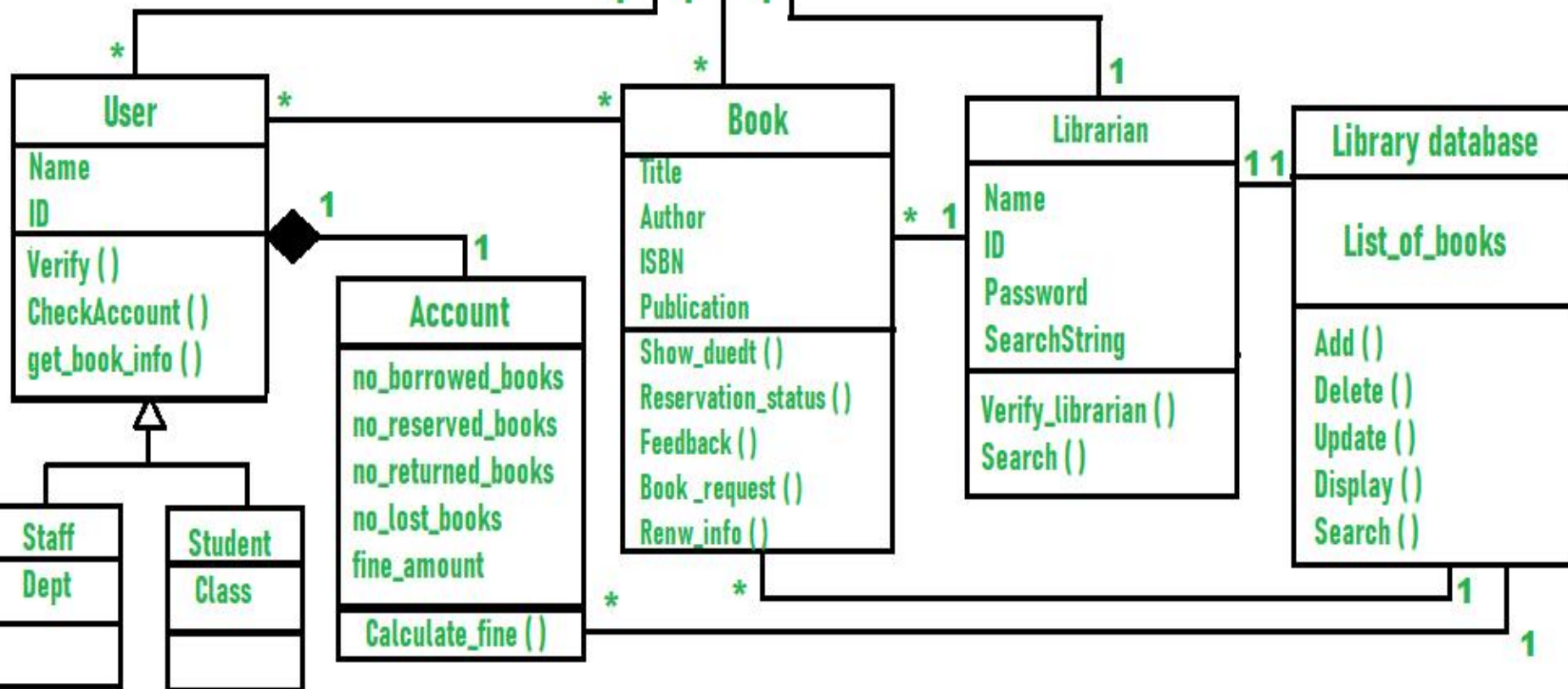
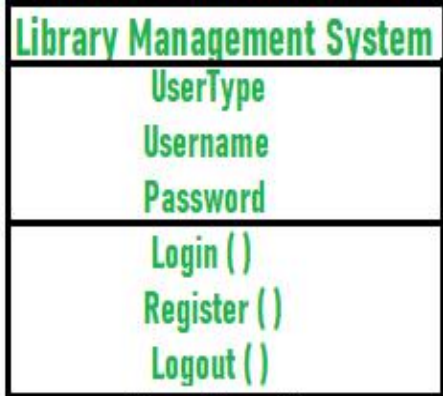
ERD Notation

One common form:



Another common form:





CLASS DIAGRAM FOR LIBRARY MANAGEMENT SYSTEM

Class-Based Modeling

- Class-based modeling represents:
 - **objects** that the system will manipulate
 - **operations** (also called methods or services) that will be applied to the objects to effect the manipulation
 - **relationships** (some hierarchical) between the objects
- The elements of a class-based model include classes and objects, attributes, operations, collaboration.

Analysis Classes

- External entities (printer, user, sensor)
- Things (report, display, signal)
- Occurrences or events (alarm, telephone call)
- Roles (manager, clerk)
- Organization units (Accounting Dept, R & D)
- Places (building, manufacturing floor)
- Structures (employee records)

Defining Attributes

- *Attributes* describe a class that has been selected for inclusion in the analysis model.
 - Build two different classes for professional baseball **players**
 - **For Playing Statistics software:** name, position, batting average, fielding percentage, years played, and games played might be relevant
 - **For Pension Fund software:** average salary, credit toward full vesting, pension plan options chosen, mailing address, and the like.
- To develop a meaningful set of attributes for an analysis class, study each use case and select those “things” that reasonably “belong” to the class.

Defining Operations

- *Operations define the behavior of an object.*
- Do a grammatical parse of a processing narrative and look at the verbs
- Operations can be divided into four broad categories:
 - (1) operations that manipulate data in some way (e.g., adding, deleting, reformatting, selecting)
 - (2) operations that perform a computation
 - (3) operations that inquire about the state of an object, and
 - (4) operations that monitor an object for the occurrence of a controlling event.

System Class

For more example of class diagram
see '[Class Diagram.pptx](#)' in the lab

System

systemID
verificationPhoneNumber
systemStatus
delayTime
telephoneNumber
masterPassword
temporaryPassword
numberTries

program()
display()
reset()
query()
arm()
disarm()

Behavioral Modeling

- The behavioral model indicates how software will respond to external events or stimuli.
- make a list of the different states of a system (How does the system behave?)
- indicate how the system makes a transition from one state to another (How does the system change state?)
 - indicate event
 - indicate action

Behavioral Modeling

- To create the model, the analyst must perform the following steps:
 - Evaluate all use-cases to fully understand the sequence of interaction within the system.
 - Identify events that drive the interaction sequence and understand how these events relate to specific objects.
 - Create a sequence for each use-case.
 - Build a state diagram for the system.
 - Review the behavioral model to verify accuracy and consistency.

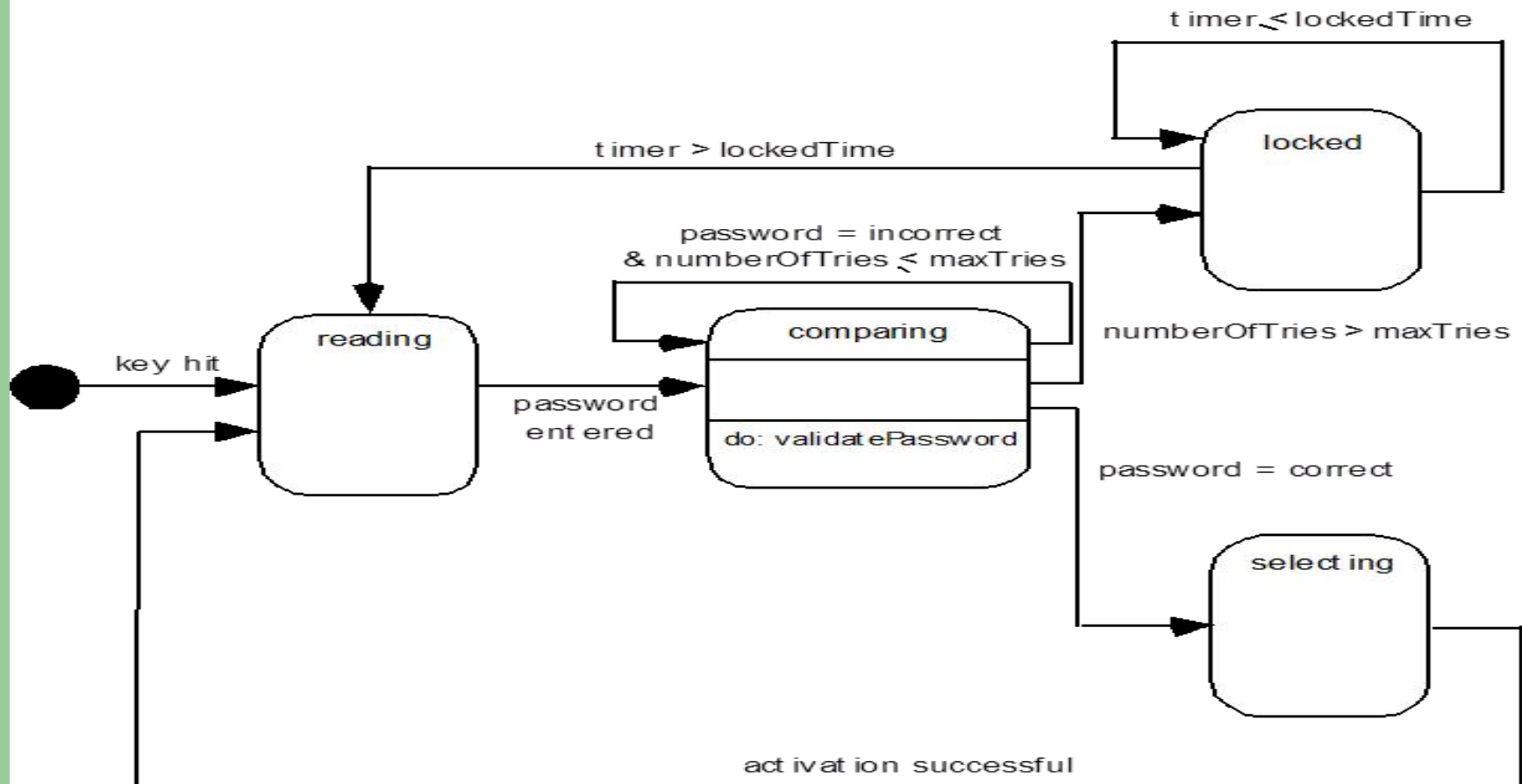
The States of a System

- **state**—a set of observable circumstances that characterizes the behavior of a system at a given time
- **state transition**—the movement from one state to another
- **event**—an occurrence that causes the system to exhibit some predictable form of behavior.
- **action**—process that occurs as a consequence of making a transition.

State Representations

- In the context of behavioral modeling, two different characterizations of states must be considered:
 - the state of each class as the system performs its function and
 - the state of the system as observed from the outside as the system performs its function
- The state of a class takes on both passive and active characteristics .
 - A *passive state* is simply the current status of all of an object's attributes.
 - The *active state* of an object indicates the current status of the object as it undergoes a continuing transformation or processing.

State Diagram for the Control Panel Class



Sequence Diagram

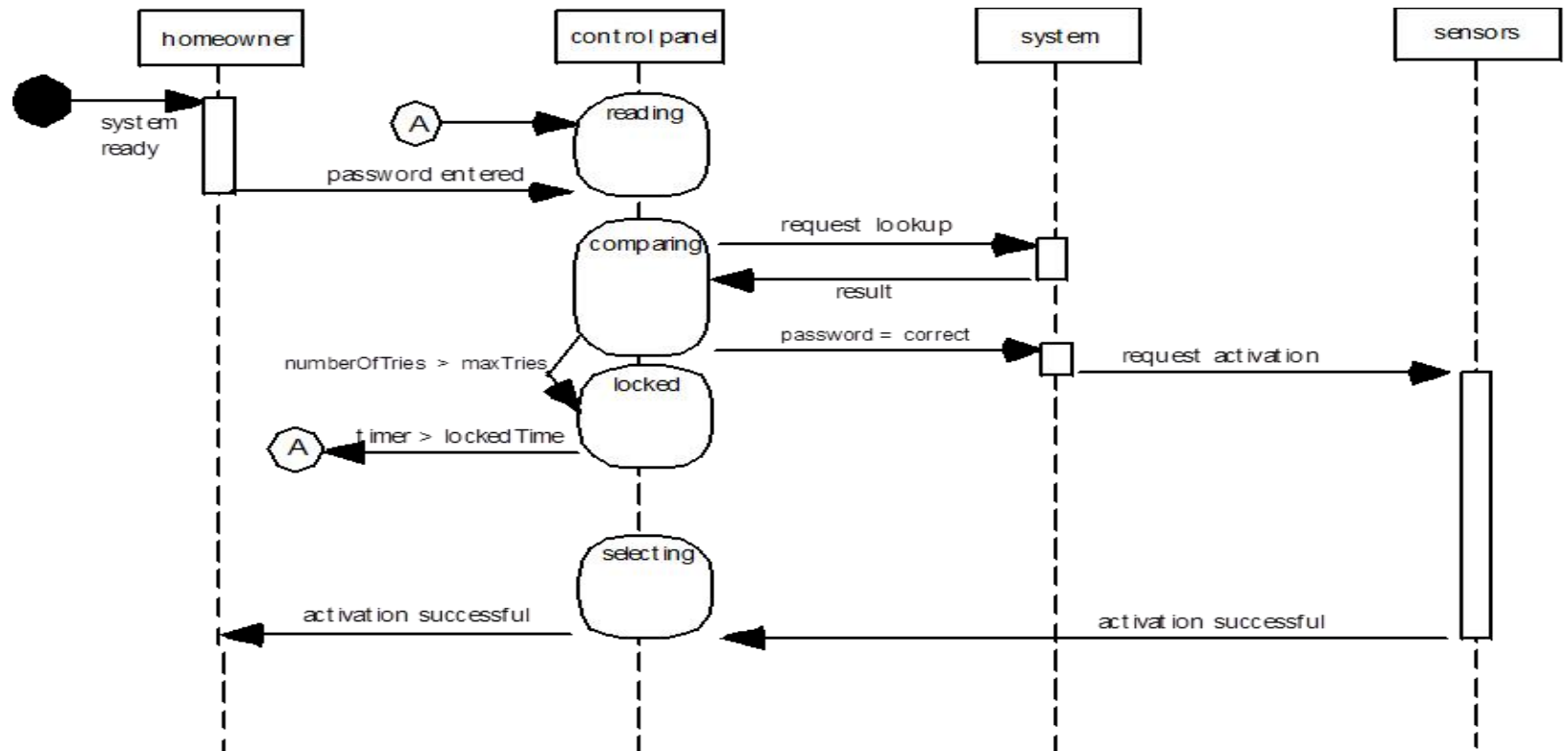


Figure 8.27 Sequence diagram (partial) for *SafeHome* security function